

How KNNs work?

Francisco Gutiérrez

What is the hypothesis set of the K-Nearest Neighbors learning model?

K-Nearest Neighbors is unusual among learning models because its hypothesis set isn't a fixed parametric family, like linear functions or a specific class of polynomials. Instead, the hypothesis set of KNN is the set of all functions that can be expressed as a majority vote, for classification, or an average, for regression, over the k nearest points in the training data, under some fixed distance metric. For a given value of k and a given distance function $d(x, x')$, the hypothesis h is defined implicitly by the training set itself:

$$h(x) = \text{mode}\{y_i : x_i \in N_k(x)\},$$

where $N_k(x)$ is the set of the k training points closest to x under d .

There are a few important implications of this setup. It is a non-parametric, instance-based hypothesis set: there's no fixed-dimensional parameter vector w that indexes the hypothesis, unlike linear regression or logistic regression. The "hypothesis" literally is the training data plus the rule. The hypothesis set is also data-dependent: change the training set, and you get a different function h , so the set of candidate hypotheses effectively grows with the size and locations of the sample points. Finally, the effective complexity is controlled by k . A small k produces hypotheses that can carve out very jagged, complicated decision boundaries, giving high complexity, low bias, and high variance. A large k produces smoother, simpler boundaries, giving low complexity, high bias, and low variance. As k approaches N , the total number of training points, the hypothesis collapses to predicting the majority class globally.

So rather than " $H = \text{all lines}$ " or " $H = \text{all degree-}d \text{ polynomials}$," KNN's hypothesis set is best thought of as the set of all piecewise-constant functions inducible by a Voronoi-like partition of the input space, for some choice of k and metric d , given a specific labeled sample.

What is its learning algorithm?

The learning algorithm is the more surprising part. KNN's "learning algorithm" is trivial: it just memorizes and stores the training data. There is no optimization step, no fitting of parameters, and no minimization of an error or loss function during training. The actual computational work happens at prediction time rather than training time. First, the entire training set $(x_1, y_1), \dots, (x_N, y_N)$ is stored, and this is the entirety of the training phase. Then, at query time, given a new point x , the algorithm computes the distance $d(x, x_i)$ from x to every stored training point, selects the k training points with the smallest distance, and produces an output. For classification, the output is the majority label among those k neighbors, with ties broken by some rule such as smallest k or random selection. For regression, the output is the average, or distance-weighted average, of their y -values.

Because of this, KNN is called a *lazy learning* algorithm, as opposed to eager learners like decision trees or SVMs, which build an explicit model during training. All the work is deferred to test time, which is why KNN training is $O(1)$ but prediction is $O(N)$ per query, or $O(\log N)$ with clever data structures like KD-trees or ball trees.